
THE EVOLUTION OF MACHINE LEARNING AND AI · WEEK 4 OF 12

Three Ways to Wire a Machine

Convolution, recurrence, attention: three guesses about the world, and what a good guess is worth

Live lecture · 2 hours · no code is run in class · everything marked ★ is quiz material

How to use this lecture

★ Red boxes = quiz material

If it is in a red box, it can be asked in the Week 6 quiz. If it is not, it cannot. There are six red boxes today, plus the list at the end.

Grey boxes = extra

Optional. For people who are curious. They point to the full booklet, which you are not required to read.

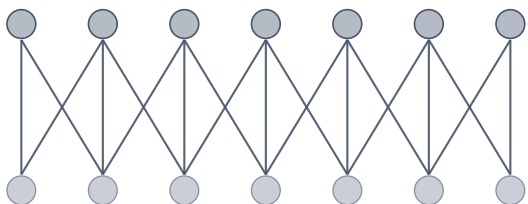
IN PLAIN ENGLISH

You need no maths beyond **add, subtract, multiply**. There are two formulas today, and both are read out in words before any symbols appear.

Your time budget this week: this lecture (2 h) · the lab (about 2 h, six exercises, one per part of today) · the **core** booklet (about 45 min). That is all. Nothing else is required.

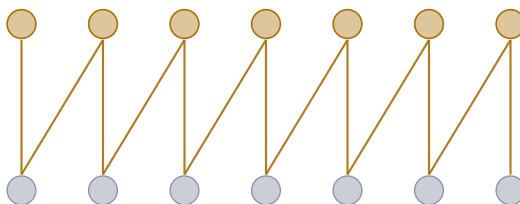
Today, in one sentence

How you wire a machine is a guess about your data. A true guess is worth a mountain of examples. A false one is worse than no guess at all.



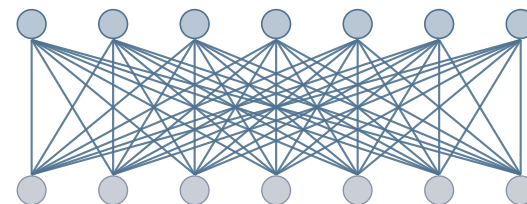
Convolution

local window,
same weights everywhere



Recurrence

a chain through time,
same weights every step



Attention

any pair may connect,
content decides which

Week 3: choosing a model is choosing a shape. This week: **which parts of the input may talk to each other at all**. Three answers, three families.

Today's route

Part 1	A guess about the world. Why a bigger model lost to a tiny one. The code, explained 1: the shuffle test.	 22 min
Part 2	Convolution: things near each other. Nine numbers, slid everywhere. The code, explained 2.	 22 min
Part 3	What the guess buys, and when it does not pay.	 12 min
Part 4	Recurrence: one thing after another. Why long chains forget. The code, explained 3.	 22 min
Part 5	Attention: who should talk to whom. The code, explained 4.	 24 min
Part 6	The bargain: a good guess is worth data, and data can replace a guess.	 11 min
Wrap	Your week, in order. What the quiz can ask.	 7 min

The one picture: three ways to run an office

EVERYDAY ANALOGY

The grid office. Everyone sits in a grid and talks only to their neighbours. To get news across the room, it is passed desk to desk. Everyone follows the same rulebook, so you train one person and photocopy them. **This is convolution.**

EVERYDAY ANALOGY

The corridor. One long line of desks. Each person gets a note from the desk before, adds what they know, passes it on. The last person answers. **This is recurrence.**

EVERYDAY ANALOGY

The open room. Anybody may talk to anybody. Before each round, everyone says what they have and what they need, so the right pairs find each other directly. **This is attention.**

KEY IDEA

None of the three is **the** right way to run an office. Which works depends on where the information sits. That is the whole lecture.

PART 1

A guess about the world

Why a bigger, cleverer model lost to a tiny one

WORDS YOU WILL HEAR IN THIS PART, IN PLAIN ENGLISH

wiring: which parts of the machine are connected to which

a guess, inductive bias: an assumption about the world, built in before any data is seen

chance: the score you get by guessing at random; here 0.25

dials: the numbers inside the machine that training adjusts (Week 3's parameters)

flexible: able to draw many different answers; not the same as good

the shuffle test: scramble the pixels the same way in every picture and see who cares

Read this list once now; each word is explained again the moment it comes up.

What comes next?

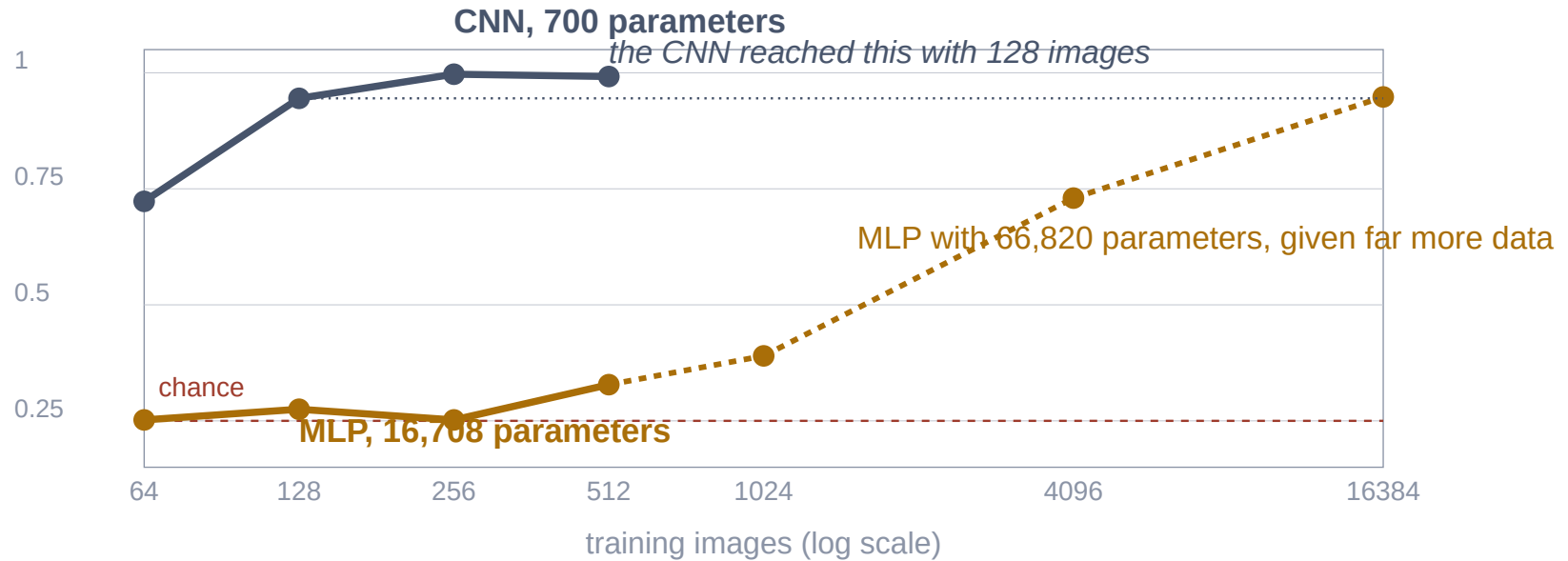
1 → 2 2 → 4 3 → 6 4 → ?

You said 8. Now defend it. Nothing in the three examples rules out “**double it, unless it is 4, then it is a thousand**”. That rule fits every example you were given.

IN PLAIN ENGLISH

Three examples fit **infinitely many** rules. Picking “double it” needed a preference that did not come from the examples: you like simple rules. That preference is called the **inductive bias**. Every learner has one, or it cannot answer “what comes next?” at all.

A tiny model beats a big one



Give the big one **95 times the data** and **128 times the pictures** and it finally catches up. **That is the exchange rate: one true guess about the world was worth about 128 times the data.**

Why did “more powerful” lose?

The big model can do **everything** the small one can, and more. It is strictly more flexible.

And it scored **1.000 on the pictures it studied**, at every data size. It was not starved of data or of training time.

IN PLAIN ENGLISH

It was starved of **reasons to prefer one answer over the others**. It could draw a million boundaries that fit the 128 pictures perfectly, and nothing told it which one was right. Flexibility is not capability. This is Week 3's memorising, with a new cause: not too much capacity, but capacity pointed in no particular direction.

The test that settles it: shuffle the pixels

Take one fixed shuffle of the 256 pixels. Apply the **same** shuffle to every picture, in training and in testing. Like cutting every photo into 256 squares and gluing them back in the same wrong order.

Nothing is lost. You could un-shuffle it. The label is still perfectly knowable. The only thing destroyed is **which pixels are neighbours**.

pixel order	tiny model (assumes neighbours)	big model (assumes nothing)	gap
original	0.992	0.328	+0.664
shuffled	0.255	0.333	-0.078

KEY IDEA

The big model did not move. The tiny model fell to **chance**. Its whole advantage was one guess about neighbours, and when the guess became false it was **worse** than assuming nothing. **A true guess is worth data. A false guess costs you.**

The code, explained 1 · the shuffle test

inductive_bias.py, the heart of it

python

```
perm = rng.permutation(256)          # one fixed shuffle
def shuffle(X):                       # same shuffle, every
    image
    return X.reshape(n, 256)[: , perm].reshape(n, 1, 16, 16)

for name, builder in (("CNN", make_cnn), ("MLP", make_mlp)):
    net = train(builder, Xtr, ytr)     # original order
    net_s = train(builder, shuffle(Xtr), ytr) # shuffled
    print(name, net.score(Xte, yte),
          net_s.score(shuffle(Xte), yte))
```

what it prints when you run it

	pixel order	CNN test	MLP test	gap
original pixel order		0.992	0.328	0.664
pixels shuffled		0.255	0.333	-0.078

WHAT EACH LINE DOES

permutation(256) picks one random new order for the 256 pixels. shuffle applies that **same** order to every picture. Then each model is trained twice, once on the real pictures and once on the shuffled ones, and scored on pictures it never saw. train is Week 3's loop; score counts the right answers.

IN THE LAB

Exercise 1 runs this (about 30 seconds), after the rule puzzle and the race at 128 pictures. The answer sheet says where to look.

Part 1, done

★ REMEMBER FOR THE WEEK 6 QUIZ

- Any set of examples fits **infinitely many** rules. Choosing one needs a preference that is not in the data: the **inductive bias**. Every learner has one.
- How a model is **wired** (which inputs may influence which outputs) is a guess about the world. A true guess means fewer dials, less data, better results on new examples.
- A more flexible model is **not** a better model. With little data it has no reason to prefer the right answer. (700 dials beat 16,708 by 67 points.)
- The **shuffle test**: destroy the thing the guess is about, and only that. If the model falls to chance, its advantage was the guess.

EXTRA · optional · NOT on the quiz · see the full booklet if curious

The formal version (hypothesis space, the no-free-lunch theorem) is in the full booklet, Chapters 1 to 3.

PART 2

Convolution: things near each other

Nine numbers, slid over the whole picture

WORDS YOU WILL HEAR IN THIS PART, IN PLAIN ENGLISH

kernel: a small square of numbers, a stencil, slid over the picture

feature map: the grid of numbers that comes out: where the stencil's pattern was found

equivariance: shift the picture and the answer shifts the same way, exactly

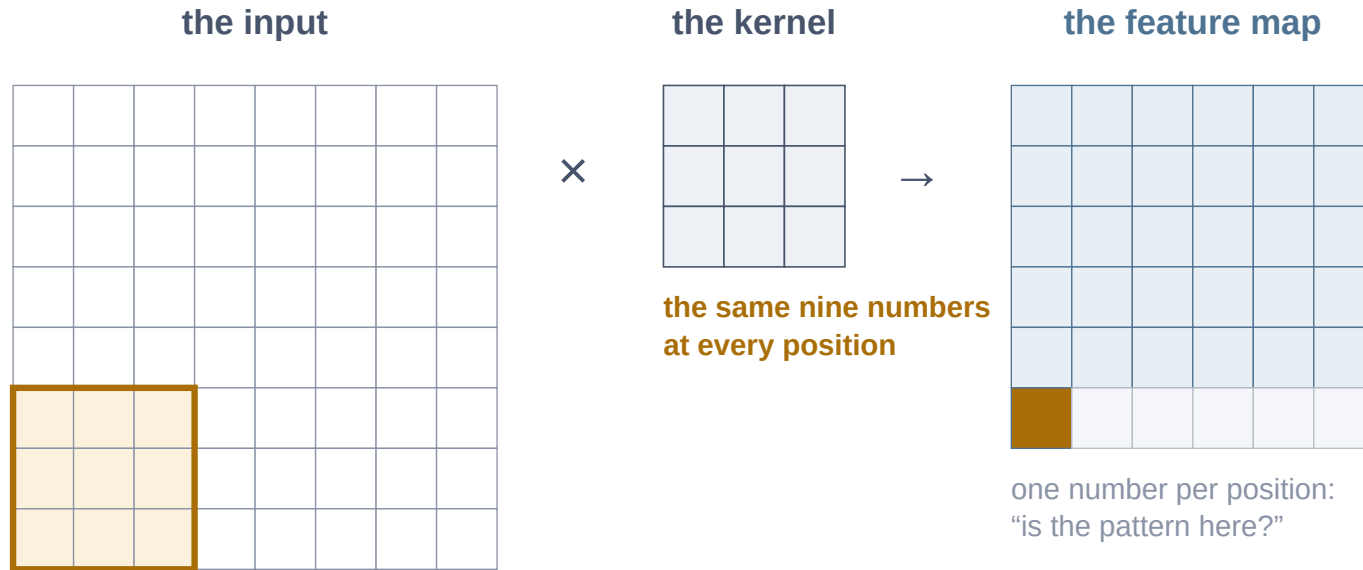
convolution: multiply the stencil by the pixels under it, add up, write one number, slide, repeat

weight sharing: the same stencil numbers used at every position

dense layer: the opposite of a stencil: every pixel has its own wire to every output

Read this list once now; each word is explained again the moment it comes up.

Watch the kernel walk



That single fact, the kernel not changing as it moves, is the whole of the next five slides.

A convolution by hand

A patch of picture, dark on the left, bright on the right:

$$\begin{pmatrix} 0.1 & 0.2 & 0.9 \\ 0.1 & 0.2 & 0.9 \\ 0.1 & 0.2 & 0.9 \end{pmatrix}$$

A kernel built to find left-to-right edges:

$$\begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$$

Now a **flat** patch, every value 0.5: the answer is **0**. The nine kernel numbers add up to zero, so a flat region gives no response. This kernel is a detector for **change**, not for brightness.

Multiply each pair, row by row, then add:

$$-0.1 + 0 + 0.9 = 0.8$$

$$-0.2 + 0 + 1.8 = 1.6$$

$$-0.1 + 0 + 0.9 = 0.8$$

Total: 3.2. A big number: an edge is here.

IN PLAIN ENGLISH

Turn the patch round (bright on the left) and the answer is **minus 3.2**. The sign tells you which way round the edge is. Two numbers you can check with a pencil, and that is the whole operation.

What a convolution assumes

THE ASSUMPTION

Things near each other belong together. What a pixel means is decided by its neighbours.

True for photos: an edge is a handful of touching pixels. False for a spreadsheet, where the column order means nothing.

THE ASSUMPTION

A pattern worth finding here is worth finding everywhere, and it looks the same. That is why the nine numbers do not change as they slide.

True for a cat's ear, which is a cat's ear in any corner of the photo.

THE ASSUMPTION

Big things are made of small things. Edges make corners, corners make parts, parts make objects.

Which is why you stack layers: each one builds on the last.

The long words, so you recognise them in a book: locality (nearby things belong together), stationarity (the same stencil everywhere), compositionality (big things are made of small things).

Move the picture and the answer moves with it

Shift the picture four pixels to the right, then run the kernel over it. Or run the kernel first, then shift the result. **You get exactly the same thing.** Not nearly the same. Exactly.

operation	difference between the two orders	verdict
convolution	0.00e+00	identical, by construction
a dense layer (every pixel wired to every output)	1.987	unrelated: it sees a new picture

IN PLAIN ENGLISH

This is called **equivariance**: move the input, the output moves the same way. A convolution has it because the same nine numbers are used everywhere, so it cannot tell where it is. A dense layer has a different wire for every pixel, so a shifted picture is simply a different picture to it.

The code, explained 2 · the operation in eight lines

convolution.py, the heart of it

python

```
def convolve2d(image, kernel):
    n = image.shape[0]; k = kernel.shape[0]
    out = np.zeros((n - k + 1, n - k + 1))
    for i in range(n - k + 1):          # every row position
        for j in range(n - k + 1):      # every column
            patch = image[i:i+k, j:j+k] # the square under
            the kernel
            out[i, j] = (patch * kernel).sum() # multiply,
            add
    return out
```

what it prints when you run it

```
Image (dark left, bright right):      Feature map:
0 0 0 1 1 1                          0 4 4 0
0 0 0 1 1 1                          0 4 4 0
0 0 0 1 1 1                          0 4 4 0
0 0 0 1 1 1                          0 4 4 0
0 0 0 1 1 1
0 0 0 1 1 1
equivariance error: 0.00e+00
```

WHAT EACH LINE DOES

Two loops walk the kernel over every position. `patch` is the square of picture under the kernel right now. `(patch * kernel).sum()` is “multiply the nine pairs and add them up”: the pencil calculation from three slides ago, done by the computer. `kernel` is never changed inside the loops: that line-that-is-not-there is weight sharing.

IN THE LAB

Exercise 2 does the pencil calculation, walks this kernel over a 6 by 6 picture, and runs the equivariance test.

Part 2, done

★ REMEMBER FOR THE WEEK 6 QUIZ

- A **convolution** slides a small square of numbers (a **kernel**) over the picture: at each position, multiply the pairs and add. The kernel **does not change** as it moves: **weight sharing**.
- It assumes three things: things near each other belong together (**locality**); a pattern worth finding here is worth finding everywhere (**stationarity**); big things are made of small things (**compositionality**).
- **Equivariance**: shift the picture and the output shifts the same way, exactly. It is a property of the arithmetic, not something learned.
- The output is a **feature map**: a map of where the kernel's pattern was found.

EXTRA · optional · NOT on the quiz · see the full booklet if curious

Stride, padding, the output-size formula and the parameter count for real images: full booklet, Chapters 6 and 7.

What the guess buys, and when it does not pay

Depth, the ending, and one honest counter-example

WORDS YOU WILL HEAR IN THIS PART, IN PLAIN ENGLISH

layer: one round of stencils; the next layer works on the previous layer's map

pooling: shrink the map by keeping one number for each small block

the ending: the last step that turns a whole map into one answer, such as "cat"

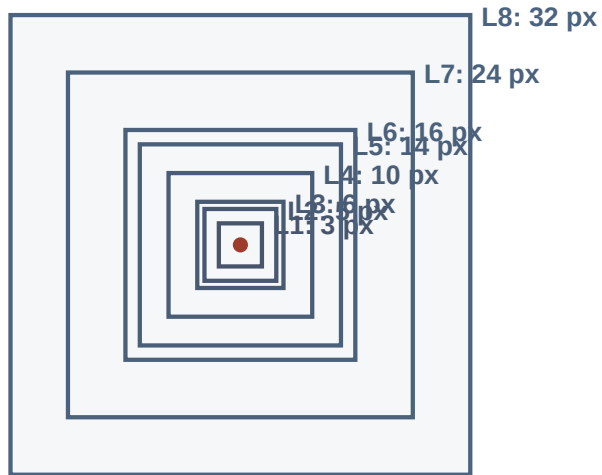
receptive field: how much of the original picture one output can "see"

invariance: move the picture and the answer does not change at all

pre-centred data: pictures where somebody already put the object in the middle for you

Read this list once now; each word is explained again the moment it comes up.

Depth turns “nearby” into “everywhere”



after	sees	pixels
L1	3×3	9
L2	5×5	25
L3	6×6	36
L4	10×10	100
L5	14×14	196
L6	16×16	256
L7	24×24	576
L8	32×32	1024

By layer 8, one unit depends on all 1,024 pixels. Nobody wrote a rule about the whole picture. Understanding of the whole was assembled out of purely local looks, like a rumour crossing the grid office desk by desk.

The ending can throw the guess away

Equivariance: move the input, the output moves. Convolution gives you this.

Invariance: move the input, the output **does not change**. This is usually what you actually want: a cat is a cat wherever it sits.

The convolution layers keep the guess. But a network needs an ending that turns a map into one answer. Choose the ending badly and **the ending throws the guess away**.

same convolution layers, two endings	dials	positions seen in training	positions never seen
ending: average the whole map	700	1.000	0.998
ending: flatten, then wire every position	2,716	0.993	0.230

KEY IDEA

Trained with the pattern only on the **left**, tested only on the **right**. Averaging does not care where: 0.998. Flattening wired a dial to every position, so “right” is new to it: chance. **A guess is worth only what your ending keeps of it.**

Where the guess fails, honestly

One. The data has no meaningful arrangement. A spreadsheet arrives pre-shuffled. **Two.** Position genuinely matters. A shadow high in the lung is a different finding from the same shadow low down, and an averaging ending cannot say so. **Three.** The things that matter are far apart. “Nearby, then nearby again” reaches far eventually, but summarises at every step. **Four.** The change is not a shift. Rotating or resizing a picture gets no guarantee at all.

MEASURED IN THE LAB

And the case nobody advertises: on small digit pictures that somebody **already centred and cropped**, the convolution model scores **0.940** and the plain model **0.978**. The plain one wins. Somebody solved “where is it?” before the model ever saw the data. **You do not get paid for a guess that was already made for you.**

Part 3, done

★ REMEMBER FOR THE WEEK 6 QUIZ

- Stacking layers turns local looks into a view of the whole picture (the **receptive field** grows). No-body writes a rule about the whole.
- **Equivariance** (output moves with the input) is not **invariance** (output does not change). You usually want invariance; the **ending** of the network decides whether you get it. Average the map: 0.998 on unseen positions. Flatten and wire every position: 0.230.
- The guess fails when the data has no arrangement, when position matters, when the important things are far apart, or when the change is not a shift.
- When the data was **pre-centred**, the guess buys nothing: the plain model won (0.978 vs 0.940).

EXTRA · optional · NOT on the quiz · see the full booklet if curious

Pooling in detail, data augmentation for rotation and scale, and the full four-experiment write-up: full booklet, Chapters 8 to 11.

Recurrence: one thing after another

The same rulebook, applied at every step in time

WORDS YOU WILL HEAR IN THIS PART, IN PLAIN ENGLISH

sequence: things that come one after another:
words in a sentence, readings over time

hidden state: the notepad: a fixed number of slots,
overwritten at every word

gate: a dial between 0 and 1 that says how much of
the memory to keep this step

recurrent network: a reader with a notepad: read a
word, update the notepad, move on

vanishing gradient: the signal from early words
shrinking to nothing through repeated multiplication

LSTM: a recurrent network with a memory and gates

Read this list once now; each word is explained again the moment it comes up.

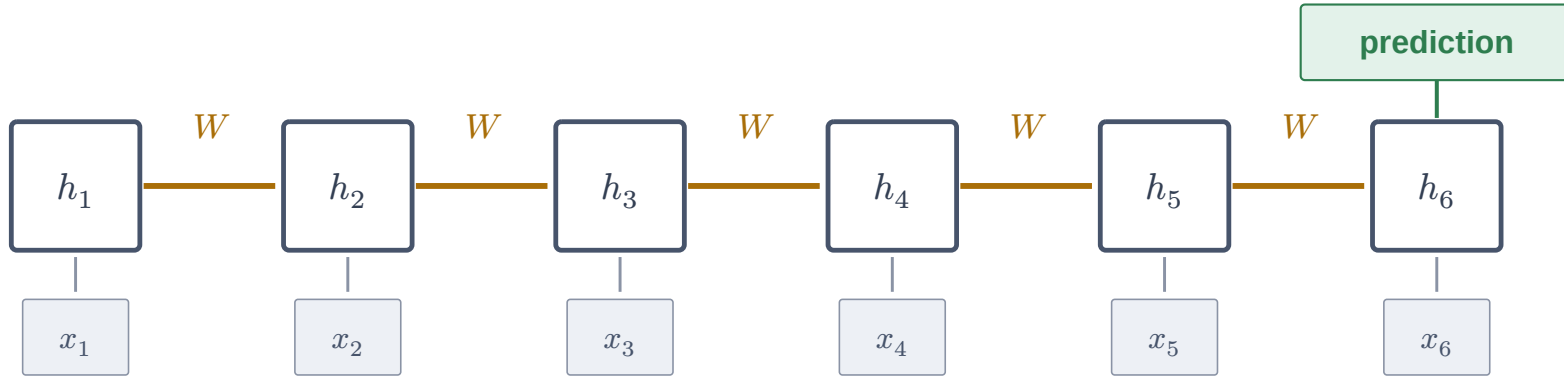
Sequences break three things at once

Length varies. Three words or three hundred. A dense layer needs a fixed size, so it cannot accept both. **Order carries the meaning.** “The dog bit the man” and “the man bit the dog” use identical words. **The things that matter can be far apart.**

The **keys** that I left on the table in the hall **are** missing.

“Are” has to agree with “keys”, eight words back, and you could make the middle as long as you like. A small window sitting at “are” sees “in”, “the”, “hall”. **It cannot see “keys” at all.**

One reader, one notepad, applied at every word



Every W is the same matrix. One layer applied six times, not six layers.

Two things follow, and they are the rest of Part 4. **The notepad is a fixed size, so there is a limit to what fits on it.** And the answer depends on word 1 only through a chain of updates that may have written over it.

Why long chains forget

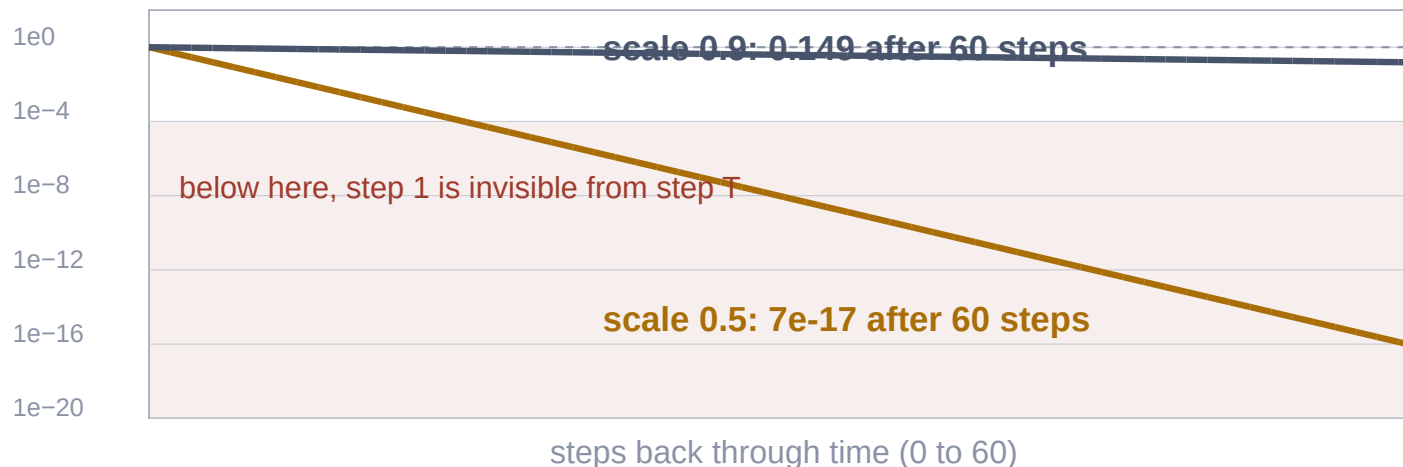
Ask: how much does word 1 still matter to the answer at word 60? The answer travels back through 60 identical updates, and at each one it is multiplied by **the same number**.

$$0.9 \times 0.9 \times \dots \text{ sixty times } = 0.0018 \qquad 1.1 \times 1.1 \times \dots \text{ sixty times } = 304$$

IN PLAIN ENGLISH

Two numbers only 0.2 apart give results **170,000 times** apart. Sixty steps of the same multiplication either shrink a message to nothing or blow it up. A deep network of ordinary layers has **different** numbers at each layer, so they can average out. A chain in time has **one** number, so there is nothing to average. This is arithmetic, not a bug.

The forgetting, measured



In a real training run, at 90 words the signal from word 1 is **7.8e-18** and the reader scores 0.463 on a task that needs word 1. At 160 words it is **1.6e-127**. **Not a typo.**

Gates: make forgetting a decision

The fix from 1997: give the reader a second notepad, the **memory**, with a **gate** on it. At each word the reader decides, fresh, how much memory to keep: 0 (wipe it) to 1 (keep it all).

The old memory is **multiplied by the gate and added to**, never pushed through the rulebook and squashed. So the signal travelling back is multiplied only by the gate, which the network **chooses**.

keep-gate	half the memory left after	1% left after
0.80	3 steps	21 steps
0.95	13 steps	90 steps
0.999	693 steps	4,603 steps

IN PLAIN ENGLISH

Keep the gate near 1 and the past survives for hundreds of steps. In a plain chain the fading rate is **one fixed number**; with gates it is a **decision**, made every step. That is the LSTM.

Gates fix the forgetting, not everything

words	plain chain: score	signal from word 1	with gates: score	signal from word 1
10	1.000	1.9e-06	1.000	1.9e-08
40	1.000	2.7e-05	0.997	5.2e-09
90	0.463	7.8e-18	1.000	8.9e-06
160	0.227	1.6e-127	0.250	7.6e-08

Row 3 is the argument. At 90 words the plain chain's signal has collapsed and it is near chance; the gated one's survives and it is perfect.

KEY IDEA

Row 4 is the more useful lesson. At 160 words the gated reader's signal **still survives**, and it **still fails**. A surviving signal is necessary, not sufficient. **The notepad is a fixed size, and 160 words' worth of "which was first?" did not fit on it.**

The code, explained 3 · one step of the reader

sequences.py, the heart of it

python

```
# plain chain: one step
def step(self, x_t, h_prev):
    return np.tanh(x_t @ self.Wx + h_prev @ self.Wh + self.b)
    # notepad = squash(word x dials + old notepad x dials)

# gated (LSTM): one step, the memory line only
f = sigmoid(x_t @ self.Wf + h_prev @ self.Uf + self.bf) #
keep-gate
c = f * c_prev + i * g          # memory = keep x old + write
x new
h = o * np.tanh(c)              # notepad read off the memory
```

what it prints when you run it

```
signal from word 1, after 60 steps (plain chain):
weight scale 0.5    6.96e-17
weight scale 0.9    1.49e-01
weight scale 1.2    4.74e+00

memory task, 90 words (chance 0.250):
plain chain  0.463    signal 7.8e-18
with gates   1.000    signal 8.9e-06
```

WHAT EACH LINE DOES

The plain step is one line: multiply the word by dials, multiply the old notepad by **the same** dials wh every time, add, squash. wh being the same at every step is why the signal is that number to the power sixty. The gated step's memory line has no squash and no wh: old memory times the keep-gate f, plus new. f is computed fresh each step from the current word.

IN THE LAB

Exercise 4 raises 0.9 and 1.1 to the sixtieth power, measures the signal through a real chain, prints the half-life table, and runs the memory task at 10, 40 and 90 words.

The two limits of the corridor

The notepad is small. Everything about the past that matters must fit on one fixed-size pad. Gates decide **what** goes on it; they do not make it **bigger**.

MEASURED IN THE LAB

A lookup task: read some pairs, then a question about one of them. As the pairs grow from 3 to 24, the gated reader goes **0.550, 0.362, 0.340, 0.278**. Chance is 0.250. The signal was healthy throughout. **The information simply did not fit.**

One desk at a time. To write notepad 1,000 you need notepad 999. A thousand steps that cannot be done at once.

KEY IDEA

Modern chips are **wide**, not fast: they do a million small sums at once, badly if they must wait for each other. A design that forbids doing things at the same time cannot use the chip. **This, not accuracy, is what decided the field.**

Part 4, done

★ REMEMBER FOR THE WEEK 6 QUIZ

- A **recurrent** network is one rulebook applied at every word: read, update a fixed-size **notepad** (the hidden state), move on. Weight sharing along time.
- Long chains forget because the signal from early words is multiplied by **the same number** at every step: 0.9 sixty times is 0.002, 1.1 sixty times is 304. The **vanishing / exploding gradient**.
- An **LSTM** adds a memory with a **keep-gate** chosen fresh each step; the memory is multiplied by the gate and added to, not squashed, so the signal can survive hundreds of steps.
- Two limits remain: the notepad is a **fixed size**, and the chain runs **one step at a time**, which modern chips hate.

EXTRA · optional · NOT on the quiz · see the full booklet if curious

The gradient product written out, all four LSTM gates, the forget-gate bias trap, and gradient clipping: full booklet, Chapters 12 to 16.

Attention: who should talk to whom

Let the words find each other directly

WORDS YOU WILL HEAR IN THIS PART, IN PLAIN ENGLISH

attention: every word looks at every other word and decides how much to listen to each

softmax: turn a row of scores into shares that add up to 1

head: one round of attention asking one question; several heads ask several

query, key, value: three cards each word writes: what I want, what I am, what I hand over

position label: a stamp on each word saying where it sits, because attention cannot see order

transformer: a machine built from blocks of attention plus a small ordinary network

Read this list once now; each word is explained again the moment it comes up.

A noticeboard, not a corridor

EVERYDAY ANALOGY

A room full of people. Each one pins two cards on a noticeboard. One says what they **are** (“I am a plumber”). One says what they **want** (“I need a plumber”). Everyone reads everyone’s cards. Where a want matches an is, those two talk directly. Nothing is relayed desk to desk.

In a sentence: the word “are” pins “I am a plural verb; I want my subject”. The word “keys” pins “I am a plural noun”. They match, across eight words, in one step.

IN PLAIN ENGLISH

The guess here is much **weaker** than the others: “any word might matter to any other, and the **content** of the words decides which”. Nothing about distance. Nothing about order. Part 6 explains why a weaker guess won anyway.

Every row adds up to 1

	the	keys	that	I	left	are
the	0.17	0.17	0.17	0.17	0.17	0.17
keys	0.05	0.38	0.05	0.08	0.05	0.38
that	0.17	0.17	0.17	0.17	0.17	0.17
I	0.14	0.21	0.14	0.15	0.14	0.21
left	0.07	0.42	0.06	0.34	0.06	0.06
are	0.02	0.74	0.01	0.12	0.01	0.1

Read the bottom row. The verb “are” gives **0.735** of its attention to “keys”, four positions back, and only **0.013** to “left”, which is right next to it.

A width-3 convolution sitting at “are” cannot see “keys” at all. Not weakly: at all.

Each row is one word asking “who should I listen to?”, and the numbers are shares that add up to 1. Read the row for **are**: 0.735 of its attention goes to **keys**, four words away, and only 0.013 to **left**, right next door. Distance played no part.

The four instructions

1. Write the cards. Each word writes three things about itself: a **query** (what I want), a **key** (what I am), a **value** (what I will hand over if chosen). Three small sets of dials do the writing. **2. Score.** Compare every query with every key. Two cards that point the same way score high. **3. Share out.** Turn each row of scores into shares that add up to 1 (this is called **softmax**). **4. Mix.** Each word's new version is the shares times the values: a blend of what the chosen words handed over.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

the same four steps, in symbols; the $\sqrt{d}$ just keeps the scores
a sensible size

Attention cannot see order

Shuffle the words, run attention, un-shuffle the outputs. Compare with running it on the original order.

input	difference	verdict
words only	8.88e-16	order is invisible: a bag of words
words + a position label	4.192	order now changes the answer

That first number is not “small”. 8.88e-16 is the rounding error of the computer’s arithmetic. The two runs are the same sums in a different order.

KEY IDEA

Bare attention **cannot tell “dog bites man” from “man bites dog”**. So every transformer ever built stamps a **position label** onto each word first. Without that stamp, it is a bag of words.

Two required additions, and the block

Position labels. Not a plain counter, which grows without limit; a set of slow and fast **clocks** whose hands give every position a unique pattern that stays small.

Several heads. One round of attention gives one blend, so a verb would have to ask for its subject, object and tense all in one question. Instead: split the width into h slices, run attention separately in each, glue the results. Same cost, h separate questions.

The transformer block. Attention (moves information **between** words), then a small ordinary network applied to each word on its own (transforms information **within** a word). Repeat the block a dozen times.

IN PLAIN ENGLISH

Attention moves information between positions; the little network changes it within a position. Alternate the two and you have every transformer you have heard of. And about two thirds of the dials are in the little ordinary networks, not in attention.

The bill

words	corridor (recurrence)	open room (attention)	ratio
64	1.7e+07	1.9e+07	1.1x
512	1.3e+08	2.7e+08	2.0x
4,096	1.1e+09	9.7e+09	9.0x
32,768	8.6e+09	5.6e+11	65x

Attention buys a path of length **1** between any two words, however far apart. It pays in sums that grow with the **square** of the length: double the words, four times the work.

This is why chatbots had short memories for years, and why so much current work is about making this table less ugly. Notice what most of those fixes do: they put a “nearby” guess back in.

The code, explained 4 · attention in eight lines

attention.py, the heart of it

python

```
def attention(Q, K, V):
    d = Q.shape[-1]
    scores = Q @ K.T / np.sqrt(d) # 2 SCORE: every query vs
    every key
    weights = softmax(scores) # 3 SHARE OUT: each row
    sums to 1
    return weights @ V # 4 MIX: shares x values

Q, K, V = E @ Wq, E @ Wk, E @ Wv # 1 WRITE THE CARDS
out = attention(Q, K, V)
```

what it prints when you run it

```
attention weights, row for 'are':
the   keys  that  I    left  are
0.02  0.73  0.01  0.12  0.01  0.10  (sum 1.00)

shuffle test:
words only           8.88e-16  order invisible
words + position label 4.192    order visible
```

WHAT EACH LINE DOES

E holds one row of numbers per word. Three multiplications write the three cards. $Q @ K.T$ compares every query with every key at once. `softmax` turns each row into shares that add to 1. $weights @ V$ blends the values. Five lines are the whole of attention, and keys wins the row for are from four words away.

IN THE LAB

Exercise 5 prints the full weights table, walks the four instructions for the word “are”, runs the shuffle test, and prints the bill.

Part 5, done

★ REMEMBER FOR THE WEEK 6 QUIZ

- **Attention** lets every word look at every other word directly; the **content** decides who listens to whom. The guess is weak: nothing about distance or order.
- Four steps: write three cards per word (**query**, **key**, **value**); **score** every query against every key; **share out** each row so it sums to 1 (**softmax**); **mix** the values by those shares.
- Bare attention **cannot see order** (a shuffled sentence gives the same answer to rounding error), so **position labels** are added first. Several **heads** ask several questions at once.
- A **transformer block** is attention (between words) followed by a small network (within a word), repeated. Cost grows with the **square** of the length.

EXTRA · optional · NOT on the quiz · see the full booklet if curious

The softmax backward pass, the sinusoidal clocks in detail, masking, and why $\sqrt{d}$: full booklet, Chapters 17 to 21.

The bargain

A good guess is worth data, and data can replace a guess

WORDS YOU WILL HEAR IN THIS PART, IN PLAIN ENGLISH

the bargain: a true guess in the wiring does the job that data would otherwise have to do

Vision Transformer: a picture chopped into squares, fed to a transformer as if they were words

crossover: the amount of data at which the machine that guesses less starts to win

scale: training on enormous amounts of data and compute

baseline: the simpler model you compare against, so you know what the clever one bought

the bitter lesson: the essay that says computation has kept beating hand-written knowledge

Read this list once now; each word is explained again the moment it comes up.

The bargain, in three lines

KEY IDEA

A guess about the world, built into the wiring, **substitutes for data**.

With **little** data: the machine whose guess matches reality wins, and wins by a lot. 700 dials beat 16,708 by 67 points. With **enormous** data: the machine that guesses less can **learn** the structure from examples, and then it wins, because it is not stuck with an approximate guess.

IN PLAIN ENGLISH

Neither line is the general rule. Which one applies to you is decided by **how much data you have**. That is the bargain.

The same architecture, the opposite conclusion

Chop a picture into small squares, treat the squares as words, run a transformer on them. Every guess about pictures is thrown away: no “nearby”, no “same everywhere”. The machine must learn from examples that touching squares are related.

1.3 million pictures

it **loses** to a convolution model

300 million pictures

it **beats** the same convolution model

Same design, same task, and the only thing that changed is the number of pictures. The lab shows the same shape on a laptop: at 128 pictures the guessing model wins by 67 points; at 128 times the data the other one has caught up.

The famous essay, read properly

Rich Sutton’s observation: for seventy years, methods that use **more computation** have kept beating methods that build in **human knowledge**.

The misreading: “knowledge is useless, just make it bigger”.

KEY IDEA

The careful reading is about where the knowledge goes. Written in as a **rule about the answer**, it tends to lose, because your rule is incomplete. Written in as a **guess about the shape of the problem**, like “nearby things belong together”, it is exactly the bargain from two slides ago, and it wins whenever you do not have 300 million pictures.

“Use more computation” is advice for organisations with hundreds of millions to spend. **For everyone else, a true guess is the cheapest data you will ever get.**

Seven questions for anything you design

- 1 What is connected to what, in one layer?
- 2 Which dials are shared, and across what: space, time, or nothing?
- 3 What does that wiring **assume**, said as a plain sentence about the world? 4 Is that sentence true of **my** data, and how would I know? The shuffle test costs one line.
- 5 Does my ending keep the guess all the way to the answer? 0.998 against 0.230. 6 What does it cost, and where is the crossover?
- 7 How much data do I have, compared with what it would take to learn the structure from scratch?

Week 1 gave seven questions for a headline. Week 2 for anything you ship. Week 3 for anything you train. **These are for anything you design.**

Part 6, done

★ REMEMBER FOR THE WEEK 6 QUIZ

- **The bargain:** a true guess built into the wiring substitutes for data. Little data: the well-guessed machine wins. Enormous data: the machine that guesses less can learn the structure and then wins.
- The **Vision Transformer** shows it: the same design loses at 1.3 million pictures and wins at 300 million. Only the data changed.
- The **bitter lesson**, read properly, is about **where** knowledge enters: as a rule about the answer it loses; as a guess about the shape of the problem it is the bargain.
- Seven questions for any design, starting with: what is wired to what, and what does that assume?

EXTRA · optional · NOT on the quiz · see the full booklet if curious

Scaling laws, the resource argument, and the case studies: full booklet, Chapters 22 to 27.

ABOUT FIVE HOURS TOTAL · DO THEM IN THIS ORDER · STOP WHEN THE LIST ENDS

Your week, in order

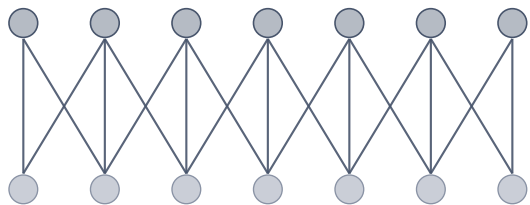
- 1 **This lecture.** You are nearly done.  2 h
- 2 **Core booklet.** Same six parts as today, with the pictures. Read once, slowly.  45 min
- 3 **The lab: six exercises, one per part of today, in this order.** The rule puzzle and the shuffle test; the convolution by hand and the kernel walk; depth, the ending and the no-payoff case; the chain that forgets and the gate; attention's four steps and the shuffle; the bargain measured.  6 × 20 min
- 4 **Check** SUBMISSION.md against your run (it is already filled in), add your name, upload it.  10 min
- + **Optional, only if you want more:** the five “break it on purpose” challenges, the three browser toys, PyTorch, the full booklet.  ∞

Every exercise names the slides it follows, tells you **what to run**, **what you should see**, and **where to look** for each answer. If you are spending more than 40 minutes on an exercise, stop and post in the channel: that is a bug in the material, not in you.

What the Week 6 quiz can ask about Week 4

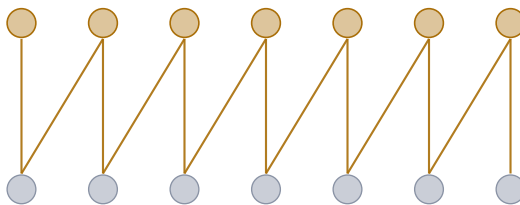
- ★1 What an inductive bias is, and why every learner must have one.
- ★2 Why a smaller model can beat a bigger one, and what the shuffle test shows.
- ★3 What a convolution does (kernel, slide, multiply, add), what it assumes, and what weight sharing is.
- ★4 Equivariance vs invariance, why depth grows the receptive field, and why the ending matters (0.998 vs 0.230).
- ★5 What a recurrent network is, why long chains forget (a number to the power T), and what the LSTM's gate changes.
- ★6 The two limits of recurrence: fixed notepad, one step at a time.
- ★7 Attention's four steps, why it cannot see order, and what position labels and heads add.
- ★8 The bargain: when a guess wins, when data wins, and the Vision Transformer as the example.

The core booklet ends with 20 practice questions **with answers**. Every question is one of these eight in disguise.



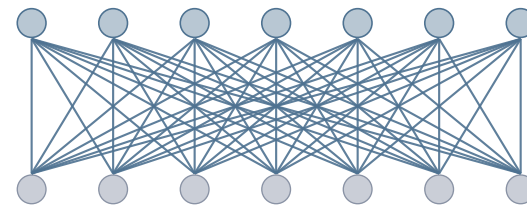
Convolution

local window,
same weights everywhere



Recurrence

a chain through time,
same weights every step



Attention

any pair may connect,
content decides which

Nearby. In order. Whoever matters. Three guesses about the world.

Questions now, or in the channel any time this week. Next week: what happens when the guess you build in is not a hunch but a law of nature.